

10.3 Pointer Subtraction and Comparison

As we've seen, you can add an integer to a pointer to get a new pointer, pointing somewhere beyond the original (as long as it's in the same array). For example, you might write

```
ip2 = ip1 + 3;
```

Applying a little algebra, you might wonder whether

```
ip2 - ip1 = 3
```

and the answer is, yes. When you subtract two pointers, as long as they point into the same array, the result is the number of elements separating them. You can also ask (again, as long as they point into the same array) whether one pointer is greater or less than another: one pointer is "greater than" another if it points beyond where the other one points. You can also compare pointers for equality and inequality: two pointers are equal if they point to the same variable or to the same cell in an array, and are (obviously) unequal if they don't. (When testing for equality or inequality, the two pointers do not have to point into the same array.)

One common use of pointer comparisons is when copying arrays using pointers. Here is a code fragment which copies 10 elements from `array1` to `array2`, using pointers. It uses an end pointer, `ep`, to keep track of when it should stop copying.

```
int array1[10], array2[10];
int *ip1, *ip2 = &array2[0];
int *ep = &array1[10];
for(ip1 = &array1[0]; ip1 < ep; ip1++)
    *ip2++ = *ip1;
```

As we mentioned, there is no element `array1[10]`, but it is legal to compute a pointer to this (nonexistent) element, as long as we only use it in pointer comparisons like this (that is, as long as we never try to fetch or store the value that it points to.)

Read sequentially: [prev](#) [next](#) [up](#) [top](#)

This page by [Steve Summit](#) // [Copyright](#) 1995-1997 // [mail feedback](#)